

Project Report: Network Bottlenecks in Virtualized Platforms

Table of Contents

I	Abstract
1.	Introduction
2.	Causes of Network Bottlenecks
3.	Identifying Bottlenecks
4.	Objectives
5.	Methodology
6.	Results & Analysis
7.	Tools & Technologies Used
	Conclusion

Abstract

In virtualized cloud environments, multiple Virtual Machines (VMs) share limited physical network resources such as NICs, virtual switches, and bandwidth. This shared resource allocation often leads to network bottlenecks, which manifest as latency, packet loss, and performance degradation.

This project simulates and analyzes network bottlenecks using Amazon Web Services (AWS) by deploying a client-server model with three EC2 instances in a Virtual Private Cloud (VPC). iperf3 is used to generate traffic, while Prometheus and Grafana provide real-time monitoring and visualization of network metrics. Bottlenecks are induced by concurrent traffic from multiple clients, and optimization strategies are applied using Linux traffic control (tc).

The results demonstrate how bottlenecks can be identified, visualized, and mitigated in virtualized platforms, offering a practical framework for cloud performance monitoring and optimization. This contributes to building more resilient, efficient, and scalable cloud systems.

1. Introduction

In virtualized platforms, multiple Virtual Machines (VMs) share the same physical resources such as NICs (Network Interface Cards), virtual switches, and bandwidth. When the collective demand exceeds resource availability, a network bottleneck occurs. This results in delays, packet loss, latency spikes, and degraded application performance.

This project demonstrates how to simulate, detect, and optimize network bottlenecks in a virtualized cloud environment using AWS EC2 instances, iperf3, Prometheus, and Grafana.

2. Causes of Network Bottlenecks

The following are the primary causes of bottlenecks in virtualized networks:

- NIC Oversubscription – Multiple VMs sharing the same NIC can exceed its bandwidth limit.
- Hypervisor Overhead – Resource allocation inefficiencies due to virtualization layer overhead.
- Virtual Switch Congestion – Internal switching delays when multiple VMs communicate simultaneously.
- Lack of QoS (Traffic Prioritization) – No prioritization for critical traffic leads to delays in high-load scenarios.

3. Identifying Bottlenecks

Bottlenecks can be detected by monitoring:

- High NIC usage (saturation of network interfaces).
- Packet loss during high throughput workloads.
- Latency spikes when bandwidth is exceeded.
- VM-to-VM traffic delays due to oversubscription.

Monitoring is achieved with Prometheus for data collection and Grafana for visualization.

4. Objectives

- To simulate network bottlenecks using multiple EC2 instances in a single VPC.
- To monitor and detect network performance issues in real-time.
- To implement traffic control techniques to optimize performance.
- To analyze the impact of bottlenecks through Prometheus + Grafana dashboards.

5. Methodology

The methodology consists of the following steps:

Step 1: Environment Setup

- Created a VPC (10.0.0.0/16) with public and private subnets.
- Launched 3 EC2 instances:
 - * Client 1 (Public Subnet) – Generates traffic.
 - * Client 2 (Public Subnet) – Generates traffic.
 - * Server (Private Subnet) – Receives traffic (iperf3 server).
- Configured Security Groups to allow SSH (22), iperf3 (5201), and Prometheus scraping (9100).

Step 2: Traffic Simulation

- Installed iperf3 on all instances.
- Configured the Server EC2 as iperf3 receiver.
- Used both Client 1 & Client 2 to send data simultaneously → caused NIC saturation on the server.

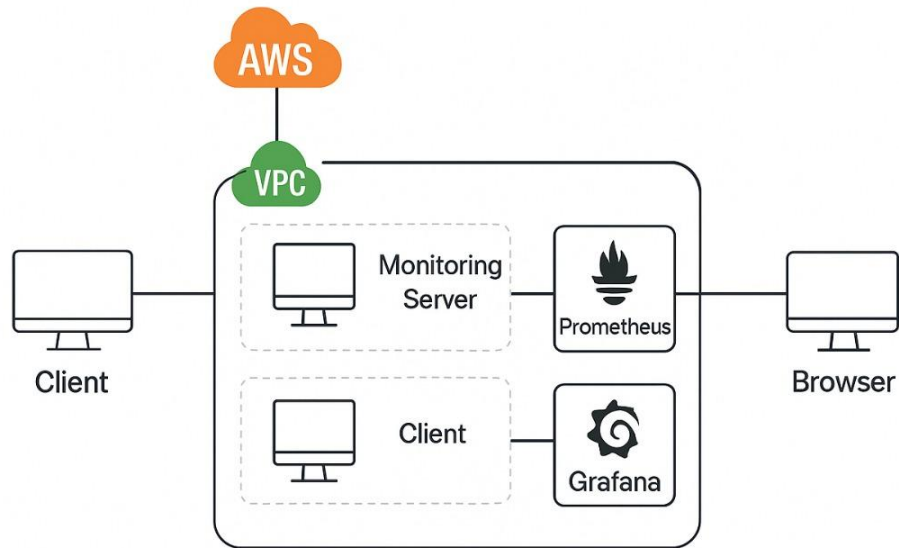
Step 3: Monitoring Setup

- Installed Prometheus on a monitoring EC2 instance.
- Installed Node Exporter for collecting CPU, memory, and network metrics.
- Integrated Grafana for visual dashboards to track:
 - * Bandwidth usage
 - * Latency trends
 - * Network throughput
 - * Packet loss

Step 4: Bottleneck Simulation & Optimization

- Observed NetworkIn spikes on the server during simultaneous client traffic.
- Applied Linux tc (Traffic Control) command on Client 1 to limit bandwidth, reducing congestion.
- Verified improvements using Prometheus metrics & Grafana visualizations.

- **System Architecture**



6. Results & Analysis

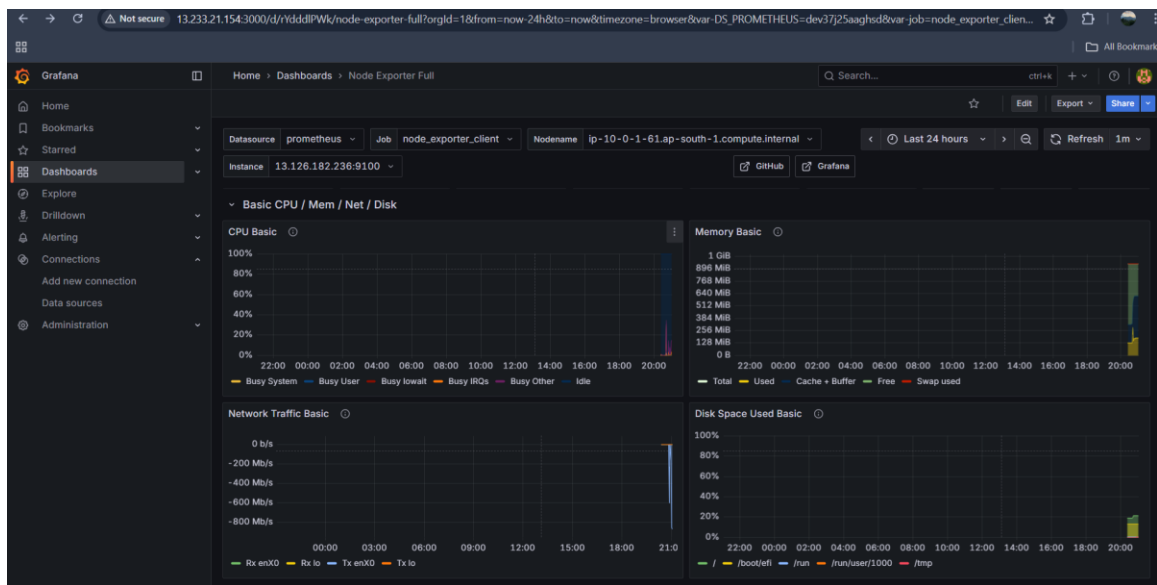
Successfully demonstrated a network bottleneck when multiple clients sent traffic concurrently.

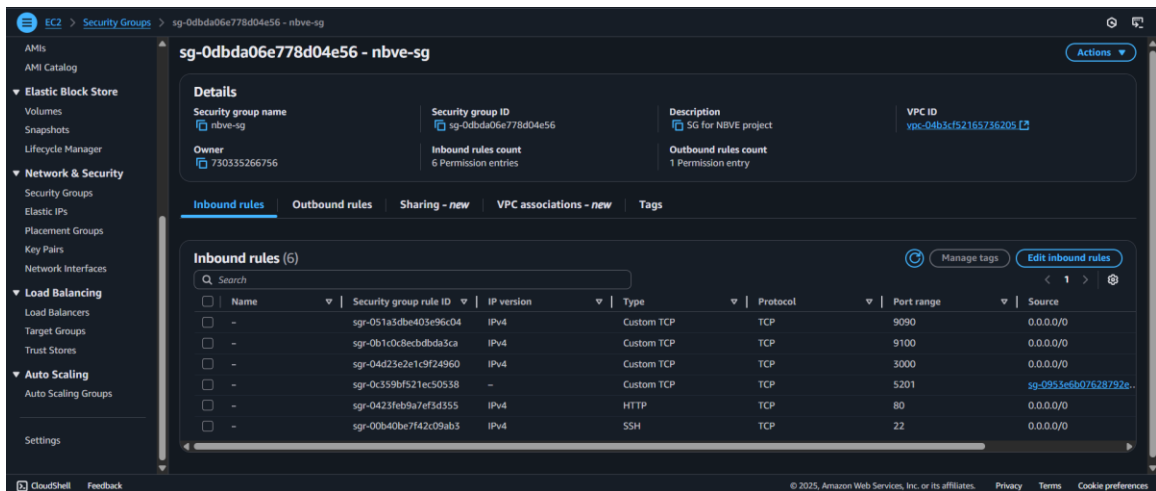
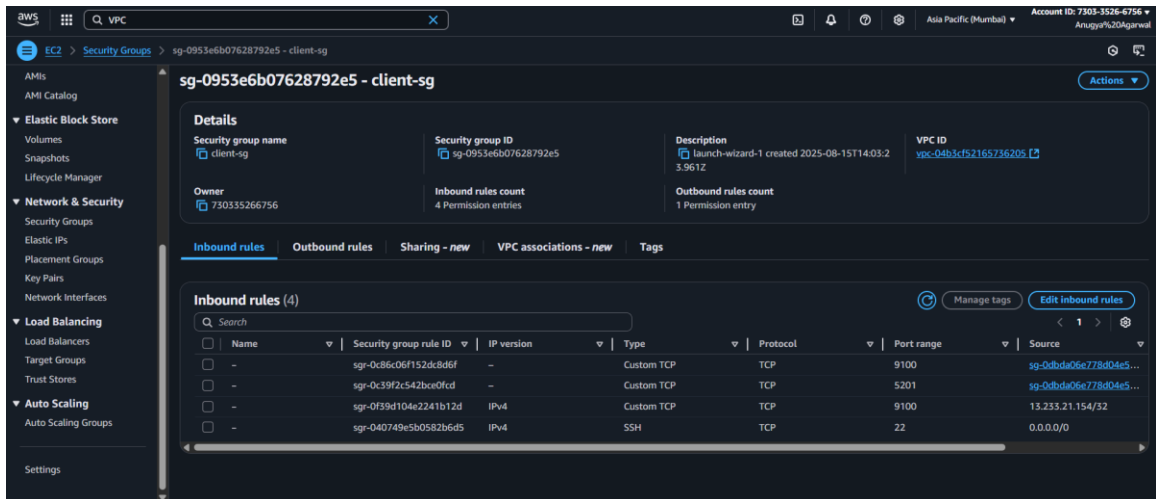
- Monitoring confirmed saturation at the server NIC, resulting in latency and packet drops.
- Traffic shaping with tc mitigated congestion, balancing network performance.
- Prometheus + Grafana dashboards provided real-time insights into performance metrics.

```

[ 5] 1.00-2.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 2.00-3.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 3.00-4.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 4.00-5.00 sec 121 MBytes 1.02 Gbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-5.00 sec 617 MBytes 1.03 Gbits/sec
-----
Server listening on 5201 (test #10)
-----
Accepted connection from 10.0.1.61, port 52856
[ 5] local 10.0.1.164 port 5201 connected to 10.0.1.61 port 52870
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00 sec 133 MBytes 1.12 Gbits/sec
[ 5] 1.00-2.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 2.00-3.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 3.00-4.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 4.00-5.00 sec 121 MBytes 1.02 Gbits/sec
[ 5] 5.00-5.01 sec 1.00 MBytes 975 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-5.01 sec 619 MBytes 1.04 Gbits/sec
-----
Server listening on 5201 (test #11)
-----
Accepted connection from 10.0.1.61, port 59242
[ 5] local 10.0.1.164 port 5201 connected to 10.0.1.61 port 59252
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00 sec 133 MBytes 1.11 Gbits/sec
[ 5] 1.00-2.00 sec 121 MBytes 1.02 Gbits/sec

```





7. Tools & Technologies Used

- AWS VPC & EC2 – Virtualized cloud environment.
- iperf3 – Traffic generation & bandwidth testing.
- Linux tc – Traffic control for congestion simulation.
- Prometheus – Metrics collection.
- Grafana – Data visualization dashboards.

AWS SETUP

Step 1 – VPC Setup

1. Go to VPC → Create VPC

- Name: nbve-vpc
- IPv4 CIDR: 10.0.0.0/16
- Tenancy: Default

Explanation: Creates an isolated network for your EC2 instances.

Step 2 – Subnet

1. Go to **Subnets** → **Create Subnet**

- Name: nbve-subnet
- VPC: nbve-vpc
- IPv4 CIDR: 10.0.1.0/24

Explanation: Subnet defines a smaller network inside the VPC for EC2 instances.

Step 3 – Internet Gateway

1. Create **Internet Gateway** → Name: nbve-igw
2. Attach to nbve-vpc

Explanation: Allows EC2 instances to access the internet for updates and package installation.

Step 4 – Route Table

1. Create **Route Table** → Name: nbve-rt
2. Attach to nbve-vpc
3. Add route: 0.0.0.0/0 → nbve-igw
4. Associate route table with nbve-subnet

Explanation: Routes all internet-bound traffic via the Internet Gateway.

Step 5 – Security Group

Security Group: nbve-sg

Protocol Port Source

Notes

Protocol	Port	Source	Notes
TCP	22	Your IP	SSH access
TCP	5000	Your IP / 0.0.0.0/0	Flask dashboard
TCP	5201	nbve-sg (self)	iperf3 client-server traffic
TCP	9100	Your IP / monitoring server IP	Prometheus metrics scraping
TCP	9200	Your IP / monitoring server IP	Python iperf exporter

- Outbound: All allowed

Explanation: Controls access to your EC2 instances and exporters.

Step 6 – Launch EC2 Instances

Name	OS	Type	Public IP	SG
monitoring-server	Amazon Linux 2023	t3.micro	Yes	nbve-sg
client-server	Amazon Linux 2023	t3.micro	Optional	nbve-sg

MONITORING SERVER SETUP

Step 1 – Update & install dependencies

```
sudo dnf update -y
```

```
sudo dnf install -y iperf3 wget tar
```

Explanation: Installs iperf3 and utilities for Prometheus/Grafana setup.

Step 2 – Install Prometheus

```
wget
```

```
https://github.com/prometheus/prometheus/releases/download/v2.53.1/prometheus-2.53.1.linux-amd64.tar.gz
```

```
tar xvf prometheus-2.53.1.linux-amd64.tar.gz
```

```
cd prometheus-2.53.1.linux-amd64
```

- **Prometheus configuration (prometheus.yml):**

scrape_configs:

- job_name: "prometheus"

static_configs:

- targets: ["localhost:9090"]

- job_name: "web-server"

static_configs:

- targets: ["<client-private-ip>:9100"] # Node Exporter

- job_name: "iperf-exporter"

static_configs:

- targets: ["<client-private-ip>:9200"] # Python iperf exporter

- Run Prometheus:

./prometheus --config.file=prometheus.yml

Explanation:

- job_name: "prometheus" → monitors Prometheus itself
- job_name: "web-server" → scrapes Node Exporter on client
- job_name: "iperf-exporter" → scrapes iperf throughput metrics from Python exporter
- Reload Prometheus after changes:

curl -X POST http://localhost:9090/-/reload

Step 3 – Install Grafana

sudo dnf install -y https://dl.grafana.com/oss/release/grafana-10.3.3-1.x86_64.rpm

sudo systemctl enable --now grafana-server

sudo systemctl start grafana-server

- Access: http://<monitoring-public-ip>:3000
- Login: admin/admin

Step 4 – Start iperf3 server

Foreground:

```
iperf3 -s
```

Background:

```
nohup iperf3 -s > iperf3_server.log 2>&1 &
```

- Verify:

```
ss -ltnp | grep 5201
```

Explanation: Receives traffic from client server.

CLIENT SERVER SETUP

Step 1 – Update & install dependencies

```
sudo dnf update -y
```

```
sudo dnf install -y iperf3 python3 git jq
```

```
python3 -m ensurepip --upgrade
```

```
python3 -m pip install --user flask psutil requests
```

Step 2 – Install Node Exporter

```
wget
```

```
https://github.com/prometheus/node\_exporter/releases/download/v1.6.1/node\_exporter-1.6.1.linux-amd64.tar.gz
```

```
tar xvf node_exporter-1.6.1.linux-amd64.tar.gz
```

```
cd node_exporter-1.6.1.linux-amd64
```

```
./node_exporter &
```

- Verify:

```
curl http://127.0.0.1:9100/metrics
```

Explanation: Exposes system metrics for Prometheus.

Step 3 – Python iperf exporter

```
mkdir -p ~/nbve
```

```
cd ~/nbve
```

```
nano iperf_exporter.py
```

Python Code:

```
#!/usr/bin/env python3
```

```
from flask import Flask, Response
```

```
import subprocess
```

```
import time
```

```
import threading
```

```
import json
```

```
app = Flask(__name__)
```

```
throughput = 0.0
```

```
RECEIVER_IP = "<monitoring-private-ip>"
```

```
throughputs = []
```

```
def run_iperf():
```

```
    global throughput
```

```
    while True:
```

```
        try:
```

```
            cmd = ['iperf3', '-c', RECEIVER_IP, '-t', '5', '-J']
```

```
            proc = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
```

```
            data = json.loads(proc.stdout)
```

```
            current = float(data['end']['sum_received']['bits_per_second']) / 1e6
```

```
            throughputs.append(current)
```

```
            if len(throughputs) > 5:
```

```
                throughputs.pop(0)
```

```
            throughput = sum(throughputs)/len(throughputs)
```

```
        except:
```

```

        throughput = 0.0

    time.sleep(1)

@app.route("/metrics")
def metrics():
    return Response(f'iperf_throughput_mbps {throughput}\n', mimetype="text/plain")

if __name__ == "__main__":
    t = threading.Thread(target=run_iperf, daemon=True)
    t.start()
    app.run(host="0.0.0.0", port=9200)

```

Step 4 – Run Python exporter

Foreground:

```
python3 iperf_exporter.py
```

Background:

```
nohup python3 iperf_exporter.py > exporter.log 2>&1 &
```

- Verify:

```
curl http://127.0.0.1:9200/metrics
```

Step 5 – Start iperf3 client

```
RECEIVER_IP=<monitoring-private-ip>
```

```
while true; do
```

```
    iperf3 -c "$RECEIVER_IP" -t 5 -J | jq -r '.end.sum_received.bits_per_second' >>
    ~/throughput_bps.log
```

```
    sleep 5
```

GRAFANA DASHBOARD SETUP

1. Open Grafana → Add Prometheus Data Source → URL: `http://localhost:9090`

2. Create Dashboard → Add Panel → Query:

`iperf_throughput_mbps`

3. Optional: Add Node Exporter metrics:

`node_network_receive_bytes_total{device="eth0"}`

`node_network_transmit_bytes_total{device="eth0"}`

`node_cpu_seconds_total`

4. Set **refresh interval** to 5s

STOP / RESTART

Client EC2

```
ps aux | grep iperf_exporter
```

```
sudo kill -9 <PID>
```

```
python3 iperf_exporter.py # restart foreground
```

Monitoring EC2

```
ps aux | grep iperf3
```

```
sudo kill -9 <PID>
```

```
nohup iperf3 -s > iperf3_server.log 2>&1 &
```

8. Conclusion

This project highlights how network bottlenecks occur in virtualized platforms and demonstrates a practical method to detect, monitor, and optimize them using AWS. The approach provides valuable insights into how cloud networking performance

can be enhanced by applying traffic shaping policies and using observability tools like Prometheus and Grafana.